

**Московский государственный технический
университет им. Н. Э. Баумана**

Факультет «Информатика и системы управления»
Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»
Отчет по лабораторной работе №6
«Работа с делегатами и рефлексией на C#»

Выполнил:
Студент группы ИУ5-35Б
Гремина Анна

Проверил:
Нардид А. Н.

2025 г.

Задание

(В качестве примера можно использовать проект «Delegates»).

1. Программа должна быть разработана в виде консольного приложения на языке C#.
2. Определите делегат, принимающий несколько параметров различных типов и возвращающий значение произвольного типа.
3. Напишите метод, соответствующий данному делегату.
4. Напишите метод, принимающий разработанный Вами делегат, в качестве одного из входным параметров. Осуществите вызов метода, передавая в качестве параметра-делегата:
 - метод, разработанный в пункте 3;
 - лямбда-выражение.
5. Повторите пункт 4, используя вместо разработанного Вами делегата, обобщенный делегат `Func< >` или `Action< >`, соответствующий сигнатуре разработанного Вами делегата.

Часть 2. Разработать программу, реализующую работу с рефлексией.

(В качестве примера можно использовать проект «Reflection»).

1. Программа должна быть разработана в виде консольного приложения на языке C#.
2. Создайте класс, содержащий конструкторы, свойства, методы.

6

-
3. С использованием рефлексии выведите информацию о конструкторах, свойствах, методах.
 4. Создайте класс атрибута (унаследован от класса `System.Attribute`).
 5. Назначьте атрибут некоторым свойствам классам. Выведите только те свойства, которым назначен атрибут.
 6. Вызовите один из методов класса с использованием рефлексии.

Листинг кода

```
1  using System;
2  using System.Reflection;
3  using System.Linq;
4
5  namespace Lab6DelegatesReflection
6  {
7      // ===== ЧАСТЬ 1: ДЕЛЕГАТЫ =====
8
9      public delegate double MathOperation(int a, double b, string operation);
10
11     public delegate void MessageDelegate(string message);
12
13     class Calculator
14     {
15         public static double Calculate(int a, double b, string operation)
16         {
17             switch (operation.ToLower())
18             {
19                 case "add":
20                     return a + b;
21                 case "multiply":
22                     return a * b;
23                 case "divide":
24                     return b != 0 ? a / b : 0;
25                 default:
26                     return 0;
27             }
28         }
29
30         public static void PrintMessage(string msg)
31         {
32             Console.WriteLine($"Сообщение: {msg}");
33         }
34
35         public static void ProcessWithDelegate(MathOperation operation, int x, double y, string op)
36         {
37             Console.WriteLine($"\\n--- Вызов через делегат как параметр ---");
38
39             double result = operation(x, y, op);
40             Console.WriteLine($"Результат операции '{op}' для ({x}, {y}): {result}");
41         }
42     }
43
44     // ===== ЧАСТЬ 2: РЕФЛЕКСИЯ =====
45
46     [AttributeUsage(AttributeTargets.Property)]
47     public class DisplayAttribute : Attribute
48     {
49         public string DisplayName { get; set; }
50
51         public DisplayAttribute(string displayName)
52         {
53             DisplayName = displayName;
54         }
55     }
56
57     public class Person
58     {
59         [Display("Имя пользователя")]
60         public string Name { get; set; }
61
62         [Display("Возраст")]
63         public int Age { get; set; }
64
65         public string Email { get; set; }
66
67         public Person()
68         {
69             Name = "Неизвестно";
70             Age = 0;
71             Email = "not_set@example.com";
72         }
73     }
```

```

73     public Person(string name, int age, string email)
74     {
75         Name = name;
76         Age = age;
77         Email = email;
78     }
79
80     public void Introduce()
81     {
82         Console.WriteLine($"Привет! Меня зовут {Name}, мне {Age} лет.");
83     }
84
85     public string GetInfo()
86     {
87         return $"Имя: {Name}, Возраст: {Age}, Email: {Email}";
88     }
89
90     private void SecretMethod()
91     {
92         Console.WriteLine("Это приватный метод!");
93     }
94 }
95
96 class Program
97 {
98     static void Main(string[] args)
99     {
100         Console.WriteLine("=== Лабораторная работа №6: Делегаты и Рефлексия ===\n");
101
102         while (true)
103         {
104             ShowMenu();
105             string? choice = Console.ReadLine();
106
107             switch (choice)

```

```

107             switch (choice)
108             {
109                 case "1":
110                     DemoDelegates();
111                     break;
112                 case "2":
113                     DemoFuncAction();
114                     break;
115                 case "3":
116                     DemoReflection();
117                     break;
118                 case "4":
119                     DemoAttributes();
120                     break;
121                 case "5":
122                     DemoReflectionInvoke();
123                     break;
124                 case "6":
125                     Console.WriteLine("Выход из программы...");
126                     return;
127                 default:
128                     Console.WriteLine("Неверный выбор. Попробуйте снова.\n");
129                     break;
130             }
131         }
132     }
133
134     static void ShowMenu()
135     {
136         Console.WriteLine("\n--- МЕНЮ ---");
137         Console.WriteLine("1. Демонстрация пользовательских делегатов");
138         Console.WriteLine("2. Демонстрация Func<> и Action<>");
139         Console.WriteLine("3. Рефлексия: информация о классе");

```

```

140 Console.WriteLine("4. Рефлексия: работа с атрибутами");
141 Console.WriteLine("5. Рефлексия: вызов методов динамически");
142 Console.WriteLine("6. Выход");
143 Console.Write("\nВыберите действие: ");
144 }
145
146 // ЧАСТЬ 1: Демонстрация делегатов
147 static void DemoDelegates()
148 {
149     Console.WriteLine("\n=== ЧАСТЬ 1: Пользовательские делегаты ===\n");
150
151     MathOperation mathOp = Calculator.Calculate;
152
153     Console.WriteLine("--- Прямой вызов делегата ---");
154     double result1 = mathOp(10, 5.5, "add");
155     Console.WriteLine($"10 + 5.5 = {result1}");
156
157     double result2 = mathOp(10, 5.5, "multiply");
158     Console.WriteLine($"10 * 5.5 = {result2}");
159
160     Calculator.ProcessWithDelegate(mathOp, 20, 4.0, "divide");
161
162     Console.WriteLine("\n--- Делегат с лямбда-выражением ---");
163     MessageDelegate msgDel = (msg) => Console.WriteLine($"[Lambda] {msg}");
164     msgDel("Привет из лямбда-выражения!");
165
166     Console.WriteLine("\n--- Многоадресный делегат ---");
167     MessageDelegate multiDel = Calculator.PrintMessage;
168     multiDel += (msg) => Console.WriteLine($"[Дополнительно] {msg}");
169     multiDel += (msg) => Console.WriteLine($"[Еще раз] {msg}");
170
171     multiDel("Тестовое сообщение");
172 }
173

```

```

174 // ЧАСТЬ 1: Демонстрация Func<> и Action<>
175 static void DemoFuncAction()
176 {
177     Console.WriteLine("\n=== ЧАСТЬ 1: Func<> и Action<> ===\n");
178
179     Console.WriteLine("--- Action<> делегат ---");
180     Action<string> printAction = (message) => Console.WriteLine($"Action: {message}");
181     printAction("Привет от Action!");
182
183     Action<int, int> sumAction = (a, b) => Console.WriteLine($"Сумма: {a + b}");
184     sumAction(5, 10);
185
186     Console.WriteLine("\n--- Func<> делегат ---");
187     Func<int, int, int> addFunc = (x, y) => x + y;
188     int sum = addFunc(7, 3);
189     Console.WriteLine($"7 + 3 = {sum}");
190
191     Func<string, int> lengthFunc = (str) => str.Length;
192     int length = lengthFunc("Hello, World!");
193     Console.WriteLine($"Длина строки 'Hello, World!': {length}");
194
195     Func<int, int> square = x => x * x;
196     Func<int, int> addTen = x => x + 10;
197
198     int value = 5;
199     int result = addTen(square(value)); // (5^2) + 10 = 35
200     Console.WriteLine($"Результат комбинации функций для 5: {result}");
201 }
202
203 // ЧАСТЬ 2: Рефлексия - информация о классе
204 static void DemoReflection()
205 {
206     Console.WriteLine("\n=== ЧАСТЬ 2: Рефлексия - Информация о классе ===\n");
207
208     Type personType = typeof(Person);
209

```

```

209 Console.WriteLine($"Полное имя класса: {personType.FullName}");
210 Console.WriteLine($"Пространство имен: {personType.Namespace}");
211 Console.WriteLine($"Имя класса: {personType.Name}");
212 Console.WriteLine($"Является ли классом: {personType.IsClass}");
213
214
215 Console.WriteLine("\n--- Конструкторы ---");
216 ConstructorInfo[] constructors = personType.GetConstructors();
217 foreach (var ctor in constructors)
218 {
219     var parameters = ctor.GetParameters();
220     string paramList = string.Join(", ", parameters.Select(p => $"{p.ParameterType.Name} {p.Name}"));
221     Console.WriteLine($"    Constructor({paramList})");
222 }
223
224 Console.WriteLine("\n--- Свойства ---");
225 PropertyInfo[] properties = personType.GetProperties();
226 foreach (var prop in properties)
227 {
228     Console.WriteLine($"    {prop.PropertyType.Name} {prop.Name}");
229     Console.WriteLine($"        Можно читать: {prop.CanRead}, Можно писать: {prop.CanWrite}");
230 }
231
232 Console.WriteLine("\n--- Методы ---");
233 MethodInfo[] methods = personType.GetMethods(BindingFlags.Public | BindingFlags.Instance | BindingFlags.DeclaredOnly);
234 foreach (var method in methods)
235 {
236     var parameters = method.GetParameters();
237     string paramList = string.Join(", ", parameters.Select(p => $"{p.ParameterType.Name} {p.Name}"));
238     Console.WriteLine($"    {method.ReturnType.Name} {method.Name}({paramList})");
239 }
240 }
241
242 // ЧАСТЬ 2: Рефлексия - работа с атрибутами

```

```

243 static void DemoAttributes()
244 {
245     Console.WriteLine("\n=== ЧАСТЬ 2: Рефлексия - Атрибуты ===\n");
246
247     Type personType = typeof(Person);
248
249     Console.WriteLine("--- Свойства с атрибутом Display ---");
250     PropertyInfo[] properties = personType.GetProperties();
251
252     foreach (var prop in properties)
253     {
254         var displayAttr = prop.GetCustomAttribute<DisplayAttribute>();
255
256         if (displayAttr != null)
257         {
258             Console.WriteLine($"    Свойство: {prop.Name}");
259             Console.WriteLine($"    Отображаемое имя: {displayAttr.DisplayName}");
260             Console.WriteLine($"    Тип: {prop.PropertyType.Name}");
261         }
262         else
263         {
264             Console.WriteLine($"    Свойство: {prop.Name} (без атрибута Display)");
265         }
266         Console.WriteLine();
267     }
268 }
269
270 // ЧАСТЬ 2: Рефлексия - вызов методов
271 static void DemoReflectionInvoke()
272 {
273     Console.WriteLine("\n=== ЧАСТЬ 2: Рефлексия - Динамический вызов методов ===\n");
274
275     Type personType = typeof(Person);
276
277     Console.WriteLine("--- Создание объекта через рефлексию ---");

```

```

278 object? personObj = Activator.CreateInstance(personType, "Аня", 20, "anya@example.com");
279
280 if (personObj == null)
281 {
282     Console.WriteLine("Ошибка создания объекта");
283     return;
284 }
285
286 Console.WriteLine("Объект создан успешно!");
287
288 Console.WriteLine("\n--- Работа со свойствами через рефлекссию ---");
289 PropertyInfo? nameProp = personType.GetProperty("Name");
290 if (nameProp != null)
291 {
292     string? currentName = nameProp.GetValue(personObj) as string;
293     Console.WriteLine($"Текущее имя: {currentName}");
294
295     nameProp.SetValue(personObj, "Анна");
296     string? newName = nameProp.GetValue(personObj) as string;
297     Console.WriteLine($"Новое имя: {newName}");
298 }
299
300 Console.WriteLine("\n--- Вызов методов через рефлекссию ---");
301 MethodInfo? introduceMethod = personType.GetMethod("Introduce");
302 if (introduceMethod != null)
303 {
304     Console.WriteLine("Вызов метода Introduce():");
305     introduceMethod.Invoke(personObj, null);
306 }
307
308 MethodInfo? getInfoMethod = personType.GetMethod("GetInfo");
309 if (getInfoMethod != null)
310 {
311     Console.WriteLine("\nВызов метода GetInfo():");
312
313     {
314         Console.WriteLine("\nВызов метода GetInfo():");
315         object? result = getInfoMethod.Invoke(personObj, null);
316         Console.WriteLine(result);
317     }
318 }
319
320 Console.WriteLine("\n--- Вызов приватного метода ---");
321 MethodInfo? secretMethod = personType.GetMethod("SecretMethod", BindingFlags.NonPublic | BindingFlags.Instance);
322 if (secretMethod != null)
323 {
324     Console.WriteLine("Вызов приватного метода SecretMethod():");
325     secretMethod.Invoke(personObj, null);
326 }
327 }
328 }

```

Результат работы программы

```
● anya@Ultrabook12978:~/Term3/lab6$ dotnet run
=== Лабораторная работа №6: Делегаты и Рефлексия ===
```

```
--- МЕНЮ ---
```

1. Демонстрация пользовательских делегатов
2. Демонстрация Func<> и Action<>
3. Рефлексия: информация о классе
4. Рефлексия: работа с атрибутами
5. Рефлексия: вызов методов динамически
6. Выход

```
Выберите действие: 1
```

```
=== ЧАСТЬ 1: Пользовательские делегаты ===
```

```
--- Прямой вызов делегата ---
```

```
10 + 5.5 = 15.5
```

```
10 * 5.5 = 55
```

```
--- Вызов через делегат как параметр ---
```

```
Результат операции 'divide' для (20, 4): 5
```

```
--- Делегат с лямбда-выражением ---
```

```
[Lambda] Привет из лямбда-выражения!
```

```
--- Многоадресный делегат ---
```

```
Сообщение: Тестовое сообщение
```

```
[Дополнительно] Тестовое сообщение
```

```
[Еще раз] Тестовое сообщение
```

```
--- МЕНЮ ---
```

1. Демонстрация пользовательских делегатов
2. Демонстрация Func<> и Action<>
3. Рефлексия: информация о классе
4. Рефлексия: работа с атрибутами
5. Рефлексия: вызов методов динамически
6. Выход

```
Выберите действие: 2
```

=== ЧАСТЬ 1: Func<> и Action<> ===

--- Action<> делегат ---

Action: Привет от Action!

Сумма: 15

--- Func<> делегат ---

7 + 3 = 10

Длина строки 'Hello, World!': 13

Результат комбинации функций для 5: 35

--- МЕНЮ ---

1. Демонстрация пользовательских делегатов
2. Демонстрация Func<> и Action<>
3. Рефлексия: информация о классе
4. Рефлексия: работа с атрибутами
5. Рефлексия: вызов методов динамически
6. Выход

Выберите действие: 3

=== ЧАСТЬ 2: Рефлексия - Информация о классе ===

Полное имя класса: Lab6DelegatesReflection.Person

Пространство имен: Lab6DelegatesReflection

Имя класса: Person

Является ли классом: True

--- Конструкторы ---

Constructor()

Constructor(String name, Int32 age, String email)

--- Свойства ---

String Name

Можно читать: True, Можно писать: True

Int32 Age

Можно читать: True, Можно писать: True

String Email

Можно читать: True, Можно писать: True

```
--- Методы ---
String get_Name()
Void set_Name(String value)
Int32 get_Age()
Void set_Age(Int32 value)
String get_Email()
Void set_Email(String value)
Void Introduce()
String GetInfo()
```

--- МЕНЮ ---

1. Демонстрация пользовательских делегатов
2. Демонстрация Func<> и Action<>
3. Рефлексия: информация о классе
4. Рефлексия: работа с атрибутами
5. Рефлексия: вызов методов динамически
6. Выход

Выберите действие: 4

=== ЧАСТЬ 2: Рефлексия - Атрибуты ===

--- Свойства с атрибутом Display ---

Свойство: Name
Отображаемое имя: Имя пользователя
Тип: String

Свойство: Age
Отображаемое имя: Возраст
Тип: Int32

Свойство: Email (без атрибута Display)

--- МЕНЮ ---

1. Демонстрация пользовательских делегатов
2. Демонстрация Func<> и Action<>
3. Рефлексия: информация о классе
4. Рефлексия: работа с атрибутами
5. Рефлексия: вызов методов динамически

6. Выход

Выберите действие: 5

=== ЧАСТЬ 2: Рефлексия - Динамический вызов методов ===

--- Создание объекта через рефлексию ---

Объект создан успешно!

--- Работа со свойствами через рефлексию ---

Текущее имя: Аня

Новое имя: Анна

--- Вызов методов через рефлексию ---

Вызов метода Introduce():

Привет! Меня зовут Анна, мне 20 лет.

Вызов метода GetInfo():

Имя: Анна, Возраст: 20, Email: anya@example.com

--- Вызов приватного метода ---

Вызов приватного метода SecretMethod():

Это приватный метод!

--- МЕНЮ ---

1. Демонстрация пользовательских делегатов
2. Демонстрация Func<> и Action<>
3. Рефлексия: информация о классе
4. Рефлексия: работа с атрибутами
5. Рефлексия: вызов методов динамически
6. Выход

Выберите действие: 6

Выход из программы...

anya@Ultrabook12978:~/Term3/lab6\$